PROJETO E ARQUITETURA DE SOFTWARE

Avaliação N2 | Arquiteturas DDD, orientada a eventos e princípios SOLID

Projeto analisado TravelBuddy	Estudante _____
Curso Engenharia de Software	Professor Luiz C. Camargo, PhD
Disciplina Projeto e Arquitetura de Software	Data da avaliação 29/10/2026

Escopo

Este documento propõe uma arquitetura DDD para o domínio do TravelBuddy, apresenta uma evolução orientada a eventos com diagramas C4 e registra seis oportunidades de melhoria SOLID identificadas no código-fonte atual. A arquitetura e as refatorações são propostas acadêmicas; não foram implementadas no runtime.

Material organizado para revisão e entrega no Teams. Preencha o nome do estudante antes do envio.

1. Domínio e arquitetura DDD

Domínio de aplicação

O TravelBuddy é uma plataforma web de compartilhamento e descoberta de experiências de viagem. Pessoas usuárias podem criar relatos associados a destinos, publicar imagens, explorar um feed e buscar conteúdo; também podem manter perfis e interagir com relatos por meio de curtidas e comentários. O domínio central é a publicação e descoberta de relatos de viagem, enquanto identidade, mídia e interações dão suporte a esse fluxo.

Linguagem ubíqua e limites de contexto

Contexto	Responsabilidade e conceitos
Identidade e Perfil	Cadastro, autenticação, credenciais, perfil e autorização. Dono dos IDs de pessoas.
Relatos de Viagem	Criação, edição, remoção, autoria, conteúdo e consulta de relatos. O código chama a entidade Destination; o nome de domínio proposto é Relato.
Interações Sociais	Curtidas e comentários, com regras próprias de unicidade e vínculo por IDs.
Descoberta	Feed, busca e projeções de leitura; consulta conteúdo sem se tornar dona dos relatos.

Agregados e dependências

- Pessoa e Relato são raízes de agregados em seus contextos; outros contextos referenciam seus IDs, evitando carregar agregados inteiros.
- Curtida é identificada por pessoa + relato. Comentário possui ID próprio e referencia ambos. Feed e busca são projeções de leitura, não fontes de verdade.
- Os contextos podem permanecer como módulos em um monólito. Interfaces HTTP chamam casos de uso; aplicação depende de portas; domínio não conhece Express, Sequelize, JWT, filesystem ou broker.

Organização-alvo

```
src/contexts/{identity-profile,travel-content,social-interaction,discovery}/
  domain/          # entidades, regras, eventos e contratos de repositório
  application/     # casos de uso e portas requeridas
  infrastructure/  # Sequelize, JWT, hash, mídia e mensageria
  interfaces/http/ # rotas, controllers, validadores e DTOs
src/shared-kernel/ # IDs e tipos realmente compartilhados
src/bootstrap/     # composição e injeção dos adaptadores
```

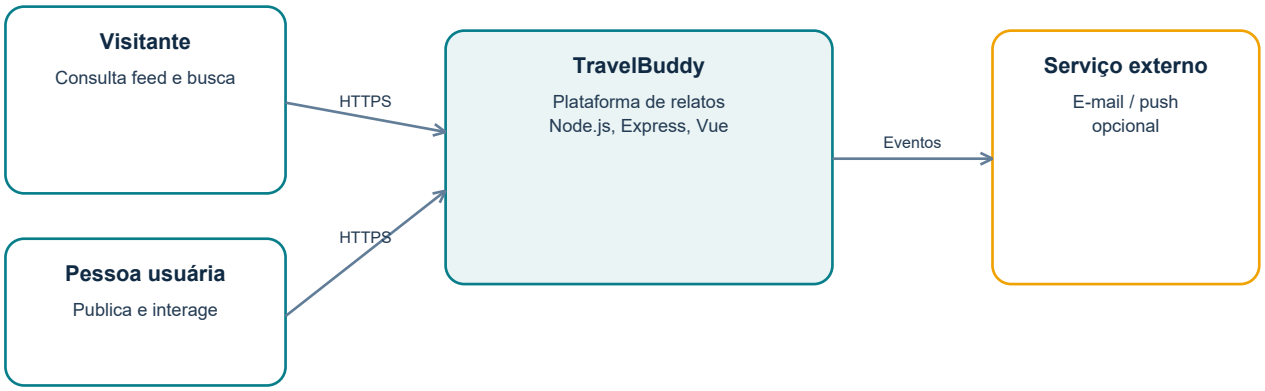
Mapeamento: modules/user → identity-profile; destination → travel-content; like/comment → social-interaction; search e feed → discovery. Esta é a arquitetura-alvo, não uma alegação de que o código atual já implemente DDD.

2. Mensageria e modelo C4

Caso de negócio

Ao publicar um novo relato, a plataforma precisa atualizar a experiência de descoberta e poderá notificar seguidores. A proposta envia `RelatoPublicado` de forma assíncrona; a requisição HTTP não espera consumidores de feed, busca ou notificações.

C4 | CONTEXTO DO SISTEMA



C4 | Contexto do sistema

O frontend Vue acessa o sistema TravelBuddy por HTTPS. A plataforma responde a visitantes e pessoas autenticadas. Serviço externo de notificação é opcional e recebe trabalho a partir de eventos, sem dependência síncrona no fluxo de publicação.

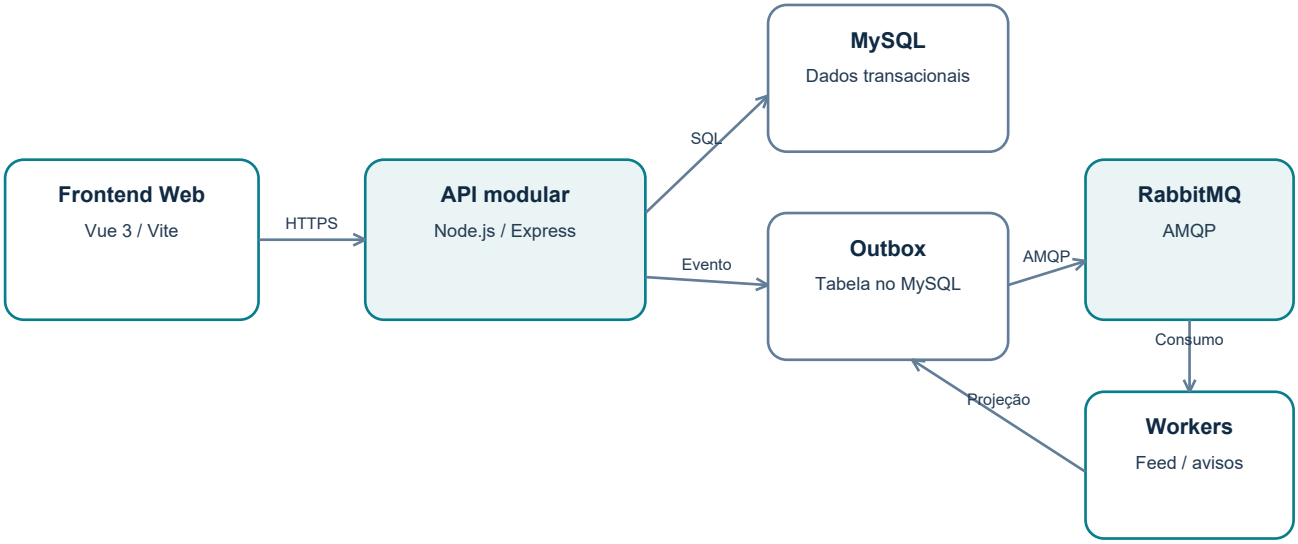
Justificativa de negócio

- Confirma a publicação sem aguardar notificações ou reconstrução da descoberta.
- Reduz dependência temporal entre publicação, busca/feed e integração de notificações; cada consumidor pode evoluir e escalar por demanda.
- Permite adicionar consumidores para novos usos de eventos do domínio sem acoplar cada função ao caso de uso de publicação.

A camada de mensageria é proposta; RabbitMQ e consumidores não estão implementados no repositório atual.

2.1 Containers e decisões técnicas

C4 | CONTAINERS COM MENSAGERIA (PROPOSTA)



C4 | Containers da solução proposta

O monólito modular continua como API. MySQL guarda os dados transacionais e uma outbox. Um relay publica eventos confirmados no RabbitMQ; workers independentes atualizam projeção de descoberta e preparam notificações.

Decisões e garantias

- RabbitMQ/AMQP: filas de trabalho e roteamento por tópicos para comunicação entre API e consumidores.
- Transactional Outbox: grava relato e evento pendente na mesma transação MySQL; relay marca envio somente após confirmação do broker.
- Entrega pelo menos uma vez: eventos carregam eventId e consumidores são idempotentes; não se promete “exactly once”.
- Contrato versionado, por exemplo eventId, eventType, version, occurredAt, relatoId e autorId; payload minimizado.
- Retry com atraso, DLQ, métricas de backlog/erro, correlationId e reprocessamento controlado. Broker em rede privada, credenciais protegidas e permissões mínimas.
- Consistência eventual: feed e busca podem refletir o relato alguns instantes depois; a interface deve considerar esse intervalo.

Fluxo: validar solicitação → transação MySQL (relato + outbox) → confirmar publicação → relay publica com confirmação → consumidores atualizam projeções e, quando aplicável, notificações.

3. Inspeção SOLID do código-fonte

Metodologia reprodutível

Inspeção estática do TravelBuddy em 23/10/2026, detalhada no backend Express: casos de uso, validação, controllers e persistência. O frontend foi considerado para entender os limites da aplicação, sem revisão exaustiva dos componentes. Para reproduzir: obter o repositório; listar `travelbuddy-backend/modules` e `middlewares`; percorrer responsabilidades, imports e efeitos colaterais; comparar as unidades com SRP e DIP; registrar caminho, linhas e trecho mínimo; sugerir refatoração; e reler as evidências no mesmo commit. Não houve alteração do runtime.

Achado 1 | SRP — serviço de relatos

Arquivo: `travelbuddy-backend/modules/destination/destinationService.js`, linhas 22-25 e 35-40.

```
async function removeFile(filename) {
  try { await fs.unlink(path.join(uploadDir, filename)); }
  catch (error) { if (error.code !== 'ENOENT') console.error(error.message); }
}
const sequelize = require('../../config/database');
const destination = await sequelize.transaction(async (transaction) => {
  const created = await Destination.create({ userId, title, description, category,
image: file.filename }, { transaction });
  await User.increment('destinationsCount', { where: { id: userId }, transaction });
  return created;
});
```

Por quê: a unidade reúne regra de relato, persistência/transação, manutenção de arquivos e logging. Mudanças em banco ou armazenamento provocam alterações no mesmo serviço.

Sugestão: caso de uso coordenador com `TravelReportRepository`, `UnitOfWork` e `MediaStorage` como portas; adaptador HTTP traduz erros de aplicação em status HTTP.

Achado 2 | SRP — serviço de usuário

Arquivo: `travelbuddy-backend/modules/user/userService.js`, linhas 51-64 e 92-116.

```
const passwordMatches = await bcrypt.compare(password, user.password);
const token = generateToken({ id: user.id, username: user.username });
// ...
const oldPath = path.join(__dirname, '..', '..', 'public/uploads/profiles', oldPicture);
fs.unlink(oldPath, (err) => { if (err) console.error(err.message); });
await user.save();
```

Por quê: autenticação, perfil, emissão de JWT e ciclo de vida da imagem mudam por motivos diferentes. Sugestão: casos de uso separados para identidade e perfil, com portas `PasswordHasher`, `TokenIssuer`, `UserRepository` e `ProfileImageStorage`.

3.1 Evidências e propostas de refatoração

Achado 3 | SRP — validação de relato

Arquivo: `travelbuddy-backend/modules/destination/destinationValidator.js`, linhas 15-23.

```
const errors = validationResult(req);
if (!errors.isEmpty()) {
  if (req.file?.path) {
    try { await require('fs/promises').unlink(req.file.path); }
    catch (cleanupError) { console.error(cleanupError.message); }
  }
  return res.status(422).json({ success: false, errors: errors.array() });
}
```

Por quê: o middleware valida entrada, remove arquivo temporário, registra erro de I/O e constrói resposta HTTP. Sugestão: manter validação no middleware, compensação de upload numa política própria e tradução central de erro no adaptador HTTP.

Achado 4 | DIP — serviço de relatos

Arquivo: `travelbuddy-backend/modules/destination/destinationService.js`, linhas 1-7 e 35-38.

```
const Destination = require('./destinationModel');
const User = require('../user/userModel');
const fs = require('fs/promises');
const path = require('path');
const sequelize = require('../../config/database');
const created = await Destination.create({ userId, title, description, category, image:
file.filename }, { transaction });
```

Por quê: a política depende diretamente de Sequelize, modelos, filesystem e caminho local. Sugestão: definir portas `TravelReportRepository`, `UserStatistics`, `UnitOfWork` e `MediaStorage`; compor adaptadores concretos no bootstrap.

DIP — regra de direção

A aplicação deve apontar para abstrações definidas pelo núcleo. Sequelize, filesystem e demais detalhes implementam essas portas na infraestrutura. Isso permite testar os casos de uso sem banco ou arquivos reais e trocar adaptadores com menor impacto.

3.2 Evidências, síntese e limites

Achado 5 | DIP — serviço de usuário

Arquivo: `travelbuddy-backend/modules/user/userService.js`, linhas 1-6 e 16-20.

```
const User = require('./userModel');
const bcrypt = require('bcryptjs');
const { generateToken } = require('../../config/jwt');
const fs = require('fs');
const Destination = require('../destination/destinationModel');
```

Por quê: o serviço depende diretamente de armazenamento, biblioteca de hash, JWT e filesystem. Sugestão: injetar `UserRepository`, `PasswordHasher`, `TokenIssuer` e `ProfileImageStorage` definidos como contratos da aplicação.

Achado 6 | DIP — serviço de comentários

Arquivo: `travelbuddy-backend/modules/comment/commentService.js`, linhas 1-4 e 13-21.

```
const sequelize = require('../../config/database');
const Comment = require('./commentModel');
const Destination = require('../destination/destinationModel');
const User = require('../user/userModel');
// cria o comentário e incrementa diretamente o modelo de relato
```

Por quê: o serviço de interação depende de modelos Sequelize de Comentário, Relato e Pessoa e atravessa outro módulo para atualizar contador. Sugestão: usar `CommentRepository`, uma porta de consulta de referência a relato e `UnitOfWork`; atualizar projeção por evento `ComentarioCriado` ou por um adaptador dono da projeção.

Síntese

Foram registrados seis achados sustentados por trechos do snapshot: três oportunidades de SRP e três de DIP. As funções e módulos foram tratados como unidades equivalentes a métodos e classes, conforme a orientação para código funcional. Os achados são oportunidades contextuais, não prova de que o sistema seja inválido; as sugestões não foram implementadas.

Limites da inspeção

Neste recorte não foi identificado caso de LSP ou ISP com evidência clara, e OCP não foi contado entre os seis para evitar atribuição especulativa. A avaliação é estática e pode mudar conforme o código evolua; caminhos e linhas devem ser conferidos se o snapshot da entrega for atualizado.

Entrega

O repositório público contém as fontes Markdown desta proposta em `docs/n2-atividade/`. Para submeter, apresente o link do repositório e envie este PDF na atividade N2 do Teams. Preencha o nome do estudante na capa antes do envio.

Projeto público: github.com/guigoEduardo/projeto_programacao_web